Brickts SQL course — Week 11 — Test like a database engineer

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 11 — Test like a database engineer

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 71: Day 71 — Assert known answers and test the tests
- Day 72: Day 72 — Test scope, constraints and row security negatively

- Day 73: Day 73 — Observe transactions, locks and bounded deadlock recovery
- Day 74: Day 74 — Freeze model revisions and copy runs consistently
- Day 75: Day 75 — Check installed schema against a versioned manifest
- Day 76: Day 76 — Replay a small failure qualification matrix
- Day 77 (weekly challenge): Day 77 — Independent challenge — an honest stress qualification packet

---

Day 71 — Assert known answers and test the tests

Previous: Day 70 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–70, or equivalent SQL skills.

Goal

Turn fixture facts into SQL assertions and show that a plausible implementation mistake is detected.

Before you start

Keep the canonical fixture intact. Use scratch views for mutations and `psql` with `ON_ERROR_STOP` for the final assertion script. Save a deliberately failing transcript separately from the passing run.

Concept

A test oracle is an independent statement of the expected result. Repeating the same formula in two queries can reproduce the same mistake twice. Derive a small expected answer from a calendar, a hand-counted interval or an explicitly constructed table. SQL assertions can raise exceptions inside a `DO` block. Mutation testing deliberately introduces a bug to check whether the assertions fail. Optional pandas may provide a testing oracle; SQL remains the primary implementation.

Tiny example

```
DO $$
BEGIN
  IF (SELECT count(*) FROM course.site) <>
        2 THEN
    RAISE EXCEPTION 'expected two
        synthetic sites';
  END IF;
  IF (SELECT count(*) FROM course.sample)
        <> 24190 THEN
    RAISE EXCEPTION 'sample fixture
        changed';
  END IF;
END $$;
```

A raised exception must make the script fail. Notices or printed booleans alone can be overlooked by CI.

SQL challenge

Assert north Supply A’s January 5 kW energy using the declared five-minute, clipped duration convention: 12 on-hours at 12 kW would be 144 kWh; one unknown five-minute kW interval leaves 143 observed kWh and five minutes unknown. Assert the missing duct-static instants separately: ten eligible samples in 08:00–09:00 UTC. Use UUID resolution and explicit site/window predicates. Compare floating results with a declared tolerance.

Experiment

Create two defective scratch versions: one replaces bad/null kW with 12, and another bridges the two missing pressure intervals as though observations existed. Show that the relevant assertions fail. Restore correct versions and rerun. Keep the expected constants independent of those mutated queries.

Acceptance checks

- The unmodified fixture passes exact counts and tolerant numeric checks.
- Each mutation triggers the intended failing assertion.
- Failure produces a nonzero script exit status.
- Coverage, unknown duration and observed energy remain separate fields.

Stretch and reflection

Which boundary case would catch an inclusive end timestamp? Design its expected answer before writing the query.

Save your SQL, actual output, and one explanation in `student-work/sql/day71/`. Use only `brickts_course`; see this week's primary

references.

Previous: Day 70 ·

Day 72 — Test scope, constraints and row security negatively

Previous: Day 71 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–71, or equivalent SQL skills.

Goal

Show that unauthorized site combinations and invalid data are rejected, and state the limits of an RLS test.

Before you start

Use scratch tables and approved existing lab accounts only. Do not create cluster roles or alter a shared server. Record the executing account's `rolsuper` and `rolbypassrls`; if no suitable restricted account exists, mark the RLS enforcement test blocked.

Concept

Composite foreign keys prevent cross-site identity combinations even when each UUID exists individually. Query predicates still need authorization derived from trusted caller context. Row-level security can add another boundary, but table owners normally bypass it unless forced, and superusers/BYPASSRLS roles bypass it. A custom session setting is not authentication when an untrusted caller can change it. PostgreSQL's row-security guide explains these account-dependent behaviors.

Tiny example

```
CREATE TABLE student.scope_demo (
  site_id text NOT NULL, amount integer
         NOT NULL
);
INSERT INTO student.scope_demo VALUES
         ('SYN-NORTH', 1), ('SYN-SOUTH',
         2);
ALTER TABLE student.scope_demo ENABLE ROW
         LEVEL SECURITY;
ALTER TABLE student.scope_demo FORCE ROW
         LEVEL SECURITY;
CREATE POLICY north_demo ON
         student.scope_demo
USING (site_id = 'SYN-NORTH') WITH CHECK
         (site_id = 'SYN-NORTH');
```

The fixed policy demonstrates filtering. A valid enforcement check requires an existing account with the necessary table privileges and no bypass. Seeing both rows as a superuser is expected bypass evidence.

SQL challenge

Build a negative-test matrix for history and ingest: south point with north site, duplicate key, invalid quality, nonfinite value, bad/null eligibility and unknown UUID. Execute data mutations only against scratch tables carrying the relevant constraints. If using an approved restricted role, verify that disallowed-site reads return zero rows and disallowed inserts or site-changing updates are rejected; record account properties and grants.

Experiment

Remove a site predicate from a scratch query. Confirm that the test detects cross-site leakage, including the repeated `Supply A` label. Then restore the predicate. Compare the result under a bypass account without mistaking it for an enforcement test.

Acceptance checks

- Every negative case has an expected outcome and actual evidence or a blocked label.
- Failed writes leave no invalid scratch rows.
- Scope tests use identity and site together.
- No role, policy or grant is installed in an application database.

Stretch and reflection

Where must trusted site authorization enter the connection/request lifecycle?

Save your SQL, actual output, and one explanation in `student-work/sql/day72/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 71 ·

Day 73 — Observe transactions, locks and bounded deadlock recovery

Previous: Day 72 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–72, or equivalent SQL skills.

Goal

Read evidence of a blocked transaction and specify a retry that replays a whole logical operation.

Before you start

Use exactly two lab sessions and a two-row scratch table. Set five-second statement deadlines and one-second lock deadlines initially. Record session PIDs and keep rollback commands ready. In psql, `\set VERBOSITY verbose` exposes SQLSTATE in error output. Touch only rows you create.

Concept

Locks protect concurrent changes, but inconsistent acquisition order can form a cycle. PostgreSQL detects a deadlock and aborts one participant with SQLSTATE `40P01`. A lock timeout produces a different failure and can prevent a cycle from lasting until detection. Canceled or failed transactions require rollback. Retrying one final statement can violate the original operation; a bounded retry must replay the whole transaction with idempotency and a consistent ordering policy.

Tiny example

```
CREATE TABLE student.lock_demo (
  id integer PRIMARY KEY, value integer
         NOT NULL
);
INSERT INTO student.lock_demo VALUES (1,
         0), (2, 0);
-- Session A:
BEGIN;
SET LOCAL lock_timeout = '1s';
SET LOCAL statement_timeout = '5s';
UPDATE student.lock_demo SET value = value
         + 1 WHERE id = 1;
```

While A holds that row, B attempts the same update with its own local deadlines. B should time out. Roll back both sessions before continuing.

SQL challenge

Capture your sessions' `pg_stat_activity`, `pg_locks` and `pg_blocking_pids` evidence for a blocked update. Identify holder, waiter, transaction state and wait duration. Then design a two-row operation that always locks IDs in ascending order. Document whole-transaction retry limits and the final expected row values after one successful operation.

Experiment

Optionally create an opposite-order cycle with the same two rows. Use a five-second statement timeout and a lock timeout long enough to permit deadlock detection. If privileges/settings prevent observing `40P01`, record the timeout actually seen. Never claim deadlock detection from an ordinary lock timeout. Roll back both sessions afterward.

Acceptance checks

- PIDs and blocking relationships belong to your lab sessions.
- SQLSTATE distinguishes lock timeout, deadlock and cancellation.
- No transaction remains idle while holding a lock.

- A retry policy caps attempts and preserves the declared logical operation.

Stretch and reflection

How would a transaction identifier or batch ledger help avoid double application after an uncertain commit?

Save your SQL, actual output, and one explanation in `student-work/sql/day73/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 72 ·

Day 74 — Freeze model revisions and copy runs consistently

Previous: Day 73 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–73, or equivalent SQL skills.

Goal

Define a reproducible run that cannot mix two model revisions or silently reuse stale cached results.

Before you start

Use invented revision identifiers and scratch tables. Keep RDF and SQL fixtures as references. No production model, cache or run is changed.

Concept

A model revision determines point-role bindings and equipment membership. A result run also depends on data window, query version, tuners and data revision. A

database snapshot can keep SQL reads consistent; it cannot automatically freeze an RDF graph held by another service. Resolve a single immutable model revision, record its identity and use an explicit SQL isolation policy. Copying a run should copy its complete child cohort atomically, not just its header. Cache keys and invalidation must account for these identities.

Tiny example

```
CREATE TABLE student.run_demo (
  run_id integer PRIMARY KEY, model_rev
         text NOT NULL,
  query_rev text NOT NULL
);
CREATE TABLE student.result_demo (
  run_id integer REFERENCES
         student.run_demo,
  equipment_id uuid NOT NULL, metric text
         NOT NULL,
  bucket timestamptz NOT NULL, value
         double precision,
  PRIMARY KEY (run_id, equipment_id,
         metric, bucket)
);
```

The foreign key prevents orphan results. It does not prove the whole intended equipment cohort was copied.

SQL challenge

Create a run containing one explicitly scoped fixture equipment cohort and its hourly metric. Design an atomic copy to a new run ID using an explicit column list. Preserve original model/query revisions and record copy provenance separately. Add assertions for child counts, equipment identities and hourly bucket keys. Demonstrate that a failure midway through the copy rolls back the new header and children together.

Experiment

In another session, change a scratch point-role selection while a reader performs two queries. Compare read-committed with a repeatable-read transaction. Then explain which external RDF state still requires a separately frozen revision. Invalidate one cached answer by changing its model or query revision.

Acceptance checks

- Every result has exactly one parent run and declared revision identities.
- The copied cohort equals the source cohort, with no duplicate or missing children.

- Failed copying leaves no partial destination run.
- The report states the SQL snapshot and external-model consistency boundary.

Stretch and reflection

Should copying a run refresh its source data? State an explicit semantic choice and its consequences for provenance.

Save your SQL, actual output, and one explanation in `student-work/sql/day74/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 73 ·

Day 75 — Check installed schema against a versioned manifest

Previous: Day 74 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–74, or equivalent SQL skills.

Goal

Detect when installed database objects differ from the schema a query and its tests expect.

Before you start

Use catalog reads and scratch migrations. Keep your source manifest small and versioned with coursework. The lab ownership sentinel identifies the dedicated database; it is not an authorization mechanism.

Concept

A migration file states intended changes; installed catalogs state what actually exists. A version number alone cannot prove columns,

constraints, policies and views match. A contract manifest can list object names, columns/types/nullability, keys, relevant constraints, view definitions and expected extension versions. Compare a canonical representation rather than unstable OIDs or generated names. CI should install from a clean database and test the resulting schema, then exercise an upgrade path where relevant.

Tiny example

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'course' AND
          table_name = 'sample'
ORDER BY ordinal_position;

SELECT schema_version, purpose
FROM course.lab_meta WHERE singleton;
```

These answer different questions: actual column properties and the declared lab schema version.

SQL challenge

Create a manifest for `course.sample`, its ownership relationships and the eligibility contract. Include the composite logical key and finite-value/quality checks. Write a catalog-based check that fails when a scratch

copy loses one expected constraint or changes a column's nullability. Distinguish a PostgreSQL type's internal name from the display form returned by `information_schema`.

Experiment

Apply a harmless scratch migration, update its expected manifest, and then simulate drift by changing only the installed schema. Confirm that the check catches the discrepancy even if a version table still claims success. Restore the scratch schema before recording the passing run.

Acceptance checks

- A clean installed schema matches the declared manifest.
- At least one missing-constraint mutation fails.
- Check output identifies the mismatched property clearly.
- Proposed CI runs SQL assertions with failure propagation and records server/extension versions.
- Migration review includes both fresh install and applicable upgrade behavior.

Stretch and reflection

Which server-generated details should be excluded from a portable manifest? Which database features demand explicit version gating?

Save your SQL, actual output, and one explanation in `student-work/sql/day75/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 74 ·

Day 76 — Replay a small failure qualification matrix

Previous: Day 75 · **Week 11 · 45–75 minutes.** Prerequisites: Days 1–75, or equivalent SQL skills.

Goal

Combine correctness and operational evidence into a repeatable, bounded qualification run.

Before you start

Reuse earlier SQL and scripts. Cap the run at two clients, forty operations, 241,900 scratch rows and a five-second statement deadline. Set a total deadline and stop conditions before starting. Do not stop shared services or inject network faults into production.

Concept

A stress result is useful only alongside correct answers, workload shape and failure accounting. A fast query returning the wrong site's data fails qualification. Replaying

failures means preserving a small synthetic input, revisions, SQL parameters, expected behavior and observed error category. A skipped cancellation test cannot become a successful cancellation claim. Recovery includes connection/transaction cleanup and removal of scratch jobs or locks.

Tiny example

```
CREATE TABLE student.qualification_case (
  case_id text PRIMARY KEY,
  expected text NOT NULL,
  observed text,
  status text NOT NULL CHECK (
    status IN ('pending', 'pass', 'fail',
         'skipped', 'blocked'))
);
INSERT INTO student.qualification_case
VALUES ('empty-window', 'zero rows, valid
         request', NULL, 'pending');
```

A case starts pending. Set its final status only after recording evidence. This ledger organizes results; it does not execute the test for you.

SQL challenge

Build a matrix covering correct history, a missing-data bucket, wrong-site scope, duplicate ingest, conflicting correction, statement cancellation, lock contention and

aggregate freshness. Reuse an independent expected answer from day71. Execute the safe cases available in your lab and attach a transcript or result artifact to each status. Write a replay note containing enough information for another learner to reproduce one failure without accessing your machine.

Experiment

Run a small normal workload, inject one bounded error, then rerun the normal query in the same session after cleanup. Compare correctness and elapsed time. Record whether the recovery path restored service within your declared deadline; do not infer that from the final query alone.

Acceptance checks

- Every case has a status, expected outcome and evidence or a specific unrun reason.
- Counts distinguish passing, failing, skipped and blocked cases.
- Recovery leaves no aborted transaction, held lab lock or scheduled scratch job.
- The report stays within its declared data and concurrency caps.

Stretch and reflection

Which unavailable test most limits your confidence? Propose a bounded environment requirement for running it later.

Save your SQL, actual output, and one explanation in `student-work/sql/day76/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 75 ·

Day 77 — Independent challenge — an honest stress qualification packet

Previous: Day 76 · **Week 11 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 71–76, or equivalent skills.

Project brief

Qualify one bounded historian and analytics design using correctness, isolation and recovery evidence. Choose the workload and failure matrix independently. Keep limits at two clients, forty operations, 241,900 synthetic rows and five seconds per statement, with a total deadline. Use only the dedicated lab.

The packet should let a reviewer reproduce the test and understand what remains unknown. References are in week 11's reading list.

Acceptance criteria

- Independent known answers cover units, scope, gaps and boundaries.
- Mutation tests demonstrate that important checks can fail.
- Negative tests cover cross-site identity and relevant constraints; RLS claims include account properties.
- Cancellation/transaction recovery and one concurrency failure have actual evidence or explicit unrun status.
- Model/query/data revisions, run-copy consistency and installed-schema checks are represented.
- Resource observations and performance claims stay within the measured workload.

Deliverables

Submit the workload definition, SQL/assertions, environment manifest, result ledger, representative plans and replay instructions for one failure. Report **pass**, **fail**, **skipped** and **blocked** separately. Neither skipped nor blocked checks count as passing. Include cleanup verification and the next necessary experiment for each unresolved requirement.

Self-review

Which failure would a happy-path test miss? What can your account bypass? Why does a low average latency fail to establish reliable service? Explain the strongest claim your evidence supports, then identify its practical limit.

Previous: Day 76 ·

Primary reading — Week 11

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 11

- PL/pgSQL assertions/errors
- Database locks
- Monitoring statistics
- pg_dump
- COPY